

Implementing Performer on Language Models

Amirreza Tarabkhah

amirreza.tarabkhah@stud.uni-heidelberg.de

Abstract

This project implements the Performer (Choromanski et al., 2020) attention mechanism in the LLAMA-3 language model and compares it with its own regular Transformer (Vaswani et al., 2017) attention mechanism. The algorithm implemented includes the Fast Attention Via positive Orthogonal Random features approach (FAVOR+) algorithm, which has the time and memory complexity of $O(L)$.

1 Introduction

Transformer models are widely used for various tasks in Natural Language Processing (NLP), such as translation, text classification, and question answering. In recent years, the size of these models has increased significantly. For instance, the number of parameters grew from 340 million in BERT-Large to 175 billion in GPT-3. The main parameter that affects the efficiency of the language model is its attention mechanism. The complexity of the attention mechanism based on the natural vanilla mechanism is $O(L^2)$. This leads to a great deal of memory and time usage, especially on long sequences. Here, to avoid this, we used Performer attention mechanism that uses (FAVOR+) algorithm.

2 Dataset

The experiments utilize the "wikitext-2-raw-v1" dataset for fine-tuning and "wikitext-103-raw-v1" for evaluation. To test performance on long sequences (exceeding 4,096 tokens), the training split of the latter dataset was repurposed as a test set. Sequences were concatenated to achieve a minimum length of 4,096. Detailed statistics for the train dataset is presented in Table 1.

As like as dataset for fine-tuning, Table 2 shows statistics for the dataset. Eventhough the dataset contains trainset, the first 100,000 parameters from

Split	#sentences
Training	36718
Validation	3760
Testing	4358

Table 1: Counts of the different data splits for wikitext-2-raw-v1

trainset used as test set to combine and make sequence length of 4,096

Split	#sentences
Training	1801350
Validation	3760
Testing	4358

Table 2: Counts of the different data splits for wikitext-103-raw-v1

3 Pretraining

The model, Llama-3.2-1B, is fine-tuned using the Performer (FAVOR+) linear attention mechanism. The fine-tuning parameters are shown in Table 3.

Name	Setting
Scheduling	Cosine with Warmup (30%)
Optimizer	Paged AdamW (32-bit)
Learning Rate	5×10^{-5}
Quantization	4-bit (NF4)
Attention	Performer (Linear)
LoRA Rank (r)	32
LoRA Alpha (α)	64
Training Steps	200
Batch Size (Eff.)	16

Table 3: Hyperparameters for the Performer attention fine-tuning

4 Algorithms

Both vanilla and Performer attention mechanism algorithms have been used here.

4.1 Vanilla Attention mechanism

The language machines natural attention mechanism described in (Vaswani et al., 2017). the result of this algorithm is a quadratic time and memory usage. This can be seen in Equation 1.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

The multiplication of the Q and K transpose causes the complexity of $O(L^2)$. Both Q and K are the matrix of $L \times R$.

4.2 Performer attention mechanism

The FAVOR+ algorithm has been implemented to implement the performer attention mechanism. The FAVOR+ algorithm described in (Choromanski et al., 2020) was used to implement this attention mechanism. Equation 2 shows how FAVOR+ works

$$\text{Attention}_{\text{FAVOR+}}(Q, K, V) = \frac{\Phi(Q) (\Phi(K)^T V)}{\Phi(Q) (\Phi(K)^T \mathbf{1})} \quad (2)$$

4.2.1 Brief look to FAVOR+

As was shown in Figure 1 and 2, the first dot product results to $r \times d$ and the final result is matrix with dimension of $L \times d$. The complexity of this equation is $O(L)$.

4.2.2 Generalized Kernelizable Attention

I transferred equation 3 to equation 4 so implementation will be easier.

$$\phi(x) = \frac{h(x)}{\sqrt{m}} [\exp(\omega_1^T x), \dots, \exp(\omega_m^T x)]^T \quad (3)$$

$$\phi(x) = \frac{1}{\sqrt{m}} \exp \left(\mathbf{W}x - \frac{\|x\|^2}{2} \mathbf{1} \right) \quad (4)$$

Then, to calculate the final output, Equation 5 (from the Performer paper Appendix) was used to calculate the output. To show it as a matrix dot product, in the code, Equation 6 has been implemented.

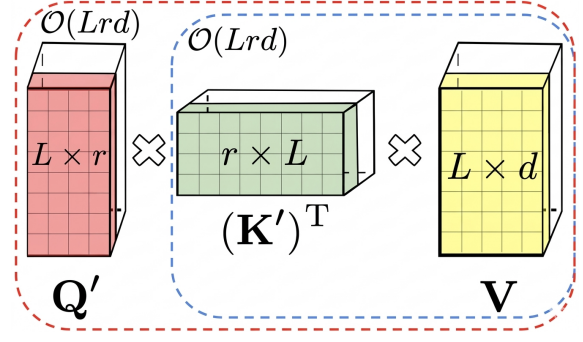


Figure 1: The Associative Property in FAVOR+ Mechanism. This diagram shows how the Performer reduces complexity. Instead of multiplying Q' and $(K')^T$ first (quadratic complexity), it first multiplies $(K')^T$ and V (blue box). This change results in linear complexity $O(Lrd)$, making it much faster for long inputs.

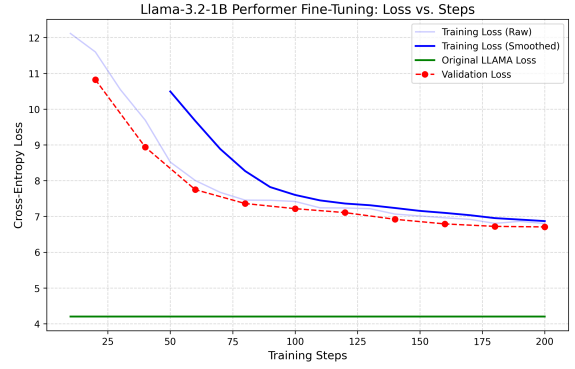


Figure 2: Loss for training and validation of Llama-3.2-1B with Performer Attention. The graph shows the cross-entropy loss over 200 training steps while fine-tuning a Llama-3.2-1B model that has been changed to use a linear FAVOR+ attention mechanism.

$$[\text{tril}(Q'(K')^T)C]_i = \mathbf{G}_{i,:}^{PS} \times \mathbf{Q}'_i, \quad \mathbf{G}_{i,:}^{PS} = \sum_{j=1}^i \mathbf{G}_{j,:}, \quad \mathbf{G}_{j,:} = \mathbf{K}'_j \mathbf{C}_j^T \quad (5)$$

$$\text{Output}_i = \frac{\phi(q_i)^T \sum_{j=1}^i (\phi(k_j) \otimes v_j)}{\phi(q_i)^T \sum_{j=1}^i \phi(k_j)} \quad (6)$$

5 Results

Cause of the need to fine-tuning language model after changing the attention mechanism, first there has been plotted the loss of fine-tuning

5.1 Fine-tuning loss

As shown in Figure 2, the training loss remains steady (from 120 to 250) and there is no need to

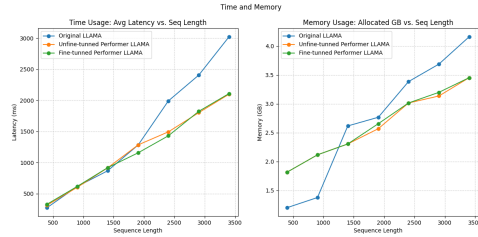


Figure 3: Comparison between Performer and Original Transformer. The left chart shows that as the sequence length increases, the Performer model is much faster (lower latency) than the Original model. The right chart shows that the Performer also uses less memory (RAM) for long sequences. Overall, the Performer is more efficient when the input size grows.

continue fine-tuning. Even with using fine-tuning after changing the attention mechanism, the loss of feedforward on original LLAMA model is less than that of the fine-tuned Performer LLAMA.

5.2 Test Results

After fine-tuning the Performer LLAMA, the original LLAMA, Fine-tuned Performer LLAMA, unfine-tuned Performer LLAMA were tested to see which one works better at time and memory consumption and in accuracy and perplexity.

5.2.1 Time and Memory consumption

By running the same testsets on each model, the result of comparing the two models is shown in Figure 3. The most important part is the sequence length of 1000 that in this length, the performance of Performer outperforms the performance of Vanilla mechanism.

It can be seen that for shorter sequence lengths (below 1000), the Performer uses slightly more memory than the Original model. This is because the Performer needs to create a "Random Feature Map", which has a fixed cost. However, as the sequence length grows larger than 1500, the Performer becomes much more efficient. While the Original model's memory usage increases very fast, the Performer's memory growth stays linear and manageable. Moreover, fine-tuning does not really affect throughput or memory consumption. Mostly, their results are the same.

5.2.2 Accuracy and Perplexity

The high perplexity and instability seen in Figure 4 is likely due to the mismatch between the pre-trained weights of Llama-3 and the Performer mechanism. Llama-3 was originally trained using

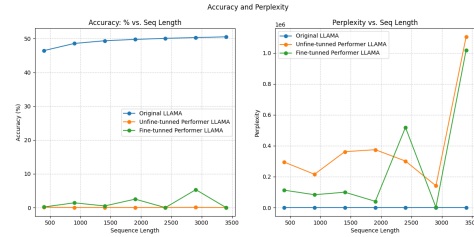


Figure 4: Performance degradation after replacing Softmax with Performer attention. Left: Accuracy (%) significantly drops for Performer-LLAMA across all sequence lengths. Right: Perplexity exhibits extreme instability and high values, indicating that pre-trained weights require adaptation to the new linear attention mechanism to regain linguistic coherence.

the standard Softmax Attention. Replacing this layer with a Linear Attention approximation introduces numerical errors that the rest of the model cannot handle.

However, the result of fine-tuning shows that the process was effective. Even with high perplexity and low accuracy, fine-tuning implementation affects the output accuracy.

6 Discussion

One of the most important points of this project is that it showed us implementing Performer on language models definitely affects the throughput and memory consumption. That means it works better in longer sequences but changing attention heads blindly, makes performance and perplexity much worse than what we expect. For better results, the model should be trained from scratch using the Performer architecture, allowing the weights to adapt to the linear attention distribution from the beginning.

References

- Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamás Szepesvári, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. 2020. [Rethinking attention with performers](#). *arXiv preprint arXiv:2009.14794*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). *arXiv preprint arXiv:1706.03762*.